

## LVM 逻辑存储

把设备容量沿 PV、VG、LV、文件系统和挂载点逐层交付，并用证据证明创建、扩展与数据保护。

对象模型 操作语义 验证 诊断 经典任务

大字号阅读版讲义

# 本章阅读导航 先抓住一条主线：应用看到的目录容量，不是直接从“磁盘”得到，而是沿着块设备、PV、VG、LV、文件系统和挂载点逐层交付。任何一层没有建立或没有同步增长，最终目录都可能得不到预期空间。

01锁定业务路径先从挂载点确认当前源设备与文件系统

02确认设备身份写入前证明目标设备没有需保留数据或冲突归属

03建立容量池PV 提供容量，VG 汇聚并按 PE 管理

04表达分配目标用 -L、-l、加号和百分比精确表达容量

05同步上层容量LV 增长后让 XFS 或 ext4 使用新增块空间

06完成分层验收分别证明 LVM、文件系统、挂载和已有数据

### 专题地图

知识专题从设备到业务目录：六层对象与容量链

知识专题PE、LE、绝对值、增量与百分比

操作专题用定制报告建立 LVM 现状图

操作专题从确认空闲的设备创建 PV、VG 和 LV

操作专题VG 空间不足时扩展容量池

操作专题保留数据扩展 LV 与文件系统

诊断专题容量没有出现时定位断裂层

诊断专题缩容、删除和已有签名的安全边界

经典任务创建指定 extent 的项目卷并扩展

经典任务保留 ext4 数据扩展现有报告卷

本章收束把容量链还原成证据链

## 阅读时持续回答 1. 当前业务目录究竟由哪个文件系统和设备提供？ 2. 目标设备是否已有文件系统、挂载、PV 或 VG 归属？ 3. 目标 VG 的 PE 多大，还剩多少可分配 extent？ 4. 题目要求的是最终总量，还是在当前基础上增加？ 5. -L 与 -l 哪个更直接表达题意？ 6. LV 已经增长后，文件系统是否也增长？ 7. 当前挂载关系是否仍指向原来的 LV？ 8. 哪条证据证明已有数据没有被重新格式化或替换？

本章边界：分区表归第 22 章，文件系统通用机制归第 23 章，fstab 持久挂载归第 24 章。本章只调用这些前置能力，不把相邻章节重新复制一遍。

## 第 25 章 · 正文

一块设备显示“还有空间”，并不等于应用已经获得可用容量。LVM 处理的是块设备层的容量组织：设备先被初始化为 PV，PV 汇入 VG，VG 再把固定大小的 extent 分配给 LV。LV 仍然只是一个块设备；上层文件系统必须增长，挂载点也必须继续指向正确源，业务目录才真正获得新增空间。

最常见的误判有四类：看到 /dev/vdb1 就假定它空闲；看到 lvextend 成功就假定目录容量已经增长；把 -L 512M 误当成“增加 512 MiB”；在遇到签名、空间不足或错误归属时，用删除、重建或强制参数绕过证据。本章不从命令清单开始，而是按“对象 → 状态 → 查询 → 修改 → 验证 → 诊断 → 安全边界”推进。

创建任务沿正向链工作：设备 → PV → VG → LV。扩容任务则应从业务路径反向定位：挂载点 → 文件系统 → LV → VG → PV。两条链最终都必须回到同一套验收：容量来源正确、对象关系正确、文件系统可见容量

正确、挂载源没有变化、原数据仍然可复核。

#### 概念

物理卷 (Physical Volume, PV) 是获得 LVM 管理身份的块设备或分区。`pvcreate` 会写入 LVM 标签和元数据, 使设备可以加入 VG; 它不是只读检测, 也不是“看到新盘就先执行”的准备动作。判断一个设备能否成为 PV, 必须先结合 `lsblk`、`findmnt`、现有签名和 `pvs` 结果确认没有需要保留的数据或冲突归属。

#### 概念

卷组 (Volume Group, VG) 把一个或多个 PV 汇聚为逻辑容量池。LV 能否继续增长, 取决于其所属 VG 是否还有空闲 extent, 而不是系统中是否存在其他磁盘或其他 VG 的剩余空间。`vgs` 中的 `vg\_free` 和 `vg\_free\_count` 分别从容量单位和 extent 数量回答“还能分配多少”。

#### 概念

逻辑卷 (Logical Volume, LV) 是从 VG 中分配出来、向上层呈现为块设备的对象。它可以承载文件系统、swap 或其他上层用途; “LV 已存在”只证明块设备层成立, 不自动证明文件系统、当前挂载或持久挂载。应从 `lvs` 读取真实路径、大小和底层映射, 不靠设备名猜测。

#### 概念

物理区域与逻辑区域 (PE/LE) 是 LVM 的固定分配单位。PE 大小属于 VG; 在线性 LV 的基本模型中, 一个 LE 对应一个 PE, 因此“PE 大小  $\times$  LE 数量”得到 LV 的标称容量。题目给出精确 extent 数量时, 直接使用 `-l` 比先换算成近似 MiB 更能表达评分目标。

#### 概念

容量池与空闲 extent 描述的是 VG 当前还能分配给 LV 的容量。底层磁盘未分区空间、新增但尚未 `pvcreate` 的设备、未加入目标 VG 的 PV, 都不是当前 LV 可直接使用的空间。扩容失败时, 下一条有区分度的证据通常是目标 VG 的 `vg\_free`、`vg\_extent\_size` 和 `vg\_free\_count`。

#### 概念

块设备容量与文件系统容量 属于两个状态层。`lvs` 读取 LV 的块设备大小, `df` 读取已挂载文件系统向用户呈现的容量; 两者因文件系统元数据和显示舍入而略有差异很正常。真正需要诊断的是 LV 明显增长而 `df` 完全不变, 这通常说明文件系统增长尚未完成。

#### 概念

挂载点到 LV 的映射 决定业务目录实际上使用哪个存储对象。目录存在或可写都不能证明挂载成立; `findmnt /srv/project` 才能确认当前源设备和文件系统类型。面对扩容任务, 应从挂载点反向找到 LV 和 VG; 若源是 NFS 或普通分区, 就不能强行套用本地 LVM 流程。

#### 概念

绝对容量与增量容量 是 `lvcreate`、`lvextend` 中必须分清的操作语义。`-L 2G` 或 `-l 70` 表示目标总量; `-L +512M` 或 `-l +20` 表示在当前基础上增加。加号不是可省略的装饰字符; 漏掉它会把“增加多少”改写成“最终达到多少”。

# 操作语义速查 进入详细专题前, 先把最关键入口放进同一张接口地图。速查区只说明命令作用对象、基本形式和最重要参数; 正文再解释调查顺序、验证边界和错误分支。

## `pvs` / `vgs` / `lvs` \*\*SYNOPSIS\*\*

```
pvs [OPTIONS] [PV ...]
vgs [OPTIONS] [VG ...]
lvs [OPTIONS] [LV ...]
```

分别读取 PV、VG、LV 的 LVM 元数据。它们是调查与验证接口，不改变目标对象；重点不是背默认列，而是用 `-o` 只显示影响当前决策的字段。 **\*\*重要参数 / 形式\*\*** `pvs -o pv\_name,vg\_name,pv\_size,pv\_free,pv\_attr`：判断哪些设备是 PV、属于哪个 VG，以及 PV 层还有多少未分配容量。 `vgs -o vg\_name,vg\_size,vg\_free,vg\_extent\_size,vg\_extent\_count,vg\_free\_count`：判断目标 VG 的总量、PE 大小和空闲 extent；这是创建与扩展 LV 前的核心容量证据。 `lvs -o lv\_name,vg\_name,lv\_size,lv\_path,lv\_attr,segtype,devices`：判断 LV 的名称、路径、大小、属性、类型和底层映射。 `--segments`：进入 segment 视图；精确 extent 题可结合 `seg\_start\_pe`、`seg\_size\_pe` 观察线性段。

### `pvcreate` **\*\*SYNOPSIS\*\***

```
pvcreate [OPTIONS] DEVICE [DEVICE ...]
```

把已确认可安全使用的块设备初始化为 PV。该操作会写入 LVM 元数据，必须位于设备身份、已有签名、挂载和数据要求调查之后。 **\*\*重要参数 / 形式\*\*** `pvcreate /dev/vdb1`：把指定设备初始化为 PV；设备名必须来自当前系统证据。 `pvs /dev/vdb1 -o pv\_name,vg\_name,pv\_size,pv\_free,pv\_attr`：操作后证明设备已成为 PV，并确认它是否已加入 VG。 `--force` / `-ff`：能绕过部分保护，不是本章默认答案。遇到已有签名或归属时先停下调查，不能用强制参数消除不确定性。

### `vgcreate` / `vgextend` **\*\*SYNOPSIS\*\***

```
vgcreate [OPTIONS] VG PV [PV ...]
vgextend [OPTIONS] VG PV [PV ...]
```

`vgcreate` 用 PV 新建容量池；`vgextend` 把新的 PV 加入已有容量池。两者改变的是 VG 的成员与可分配容量，不直接扩大任何文件系统。 **\*\*重要参数 / 形式\*\*** `vgcreate -s 16M projectvg /dev/vdb1`：创建 `projectvg`，并把 PE 大小设为 16 MiB；`-s` 是 VG 属性。 `vgextend projectvg /dev/vdc1`：把已初始化为 PV 的设备加入目标 VG；参数顺序是“目标 VG + 新 PV”。 `vgs projectvg -o vg\_name,pv\_count,vg\_size,vg\_free,vg\_extent\_size,vg\_free\_count`：创建或扩展后重新证明成员数量、PE 和空闲容量。

### `lvcreate` **\*\*SYNOPSIS\*\***

```
lvcreate [OPTIONS] -n LV VG
```

从 VG 的空闲 extent 中创建 LV。命令必须直接表达题目给出的容量方式，并在创建后验证真实路径和大小。 **\*\*重要参数 / 形式\*\*** `-n NAME`：指定 LV 名称，例如 `-n projectlv`。 `-L SIZE`：按容量创建，例如 `-L 800M` 或 `-L 2G`。 `-l EXTENTS`：按 extent 数量创建，例如 `-l 50`。 `-l 100%FREE`：使用 VG 当前全部空闲 extent；只有题目明确要求占满余量时才使用。

### `lvextend` **\*\*SYNOPSIS\*\***

```
lvextend [OPTIONS] LV
```

从目标 VG 的空闲 extent 中扩大已有 LV。它不会自动发现新磁盘，也不会借用其他 VG 的容量；空间不足时应先调查并扩展 VG。 **\*\*重要参数 / 形式\*\*** ``-L 3G``: 把 LV 的目标总大小设为 3 GiB。 ``-L +512M``: 在当前基础上增加 512 MiB。 ``-l 70``: 把目标总 extent 数设为 70。 ``-l +20``: 在当前基础上增加 20 个 extent。 ``-l +100%FREE``: 把 VG 当前全部空闲 extent 追加给已有 LV。 ``-r`` / ``--resizefs``: 在 LV 扩展后调用文件系统调整工具。它跨越 LV 和文件系统两个状态层，操作后仍要分别查询。

```
## `xfs_growfs` / `resize2fs` **SYNOPSIS**
```

```
xfs_growfs [OPTIONS] MOUNT_POINT
resize2fs [OPTIONS] DEVICE [SIZE]
```

在底层 LV 已经扩大后，让文件系统使用新增块空间。必须先确认文件系统类型；不能用重新格式化代替增长。 **\*\*重要参数 / 形式\*\*** ``xfs_growfs /srv/project``: 扩大当前挂载在 ``/srv/project`` 的 XFS。XFS 在 RHEL 9 主线中支持增长，不支持原地缩小。 ``resize2fs /dev/datavg/reportslv``: 扩大 ext4 文件系统；不指定大小时通常增长到当前块设备可用上限。 ``lvextend -r ...``: 可把 LV 与受支持文件系统的增长串联，但任何失败后都要重新查询实际中间状态。

```
## `findmnt` / `lsblk` / `df` **SYNOPSIS**
```

```
findmnt [OPTIONS] [DEVICE|MOUNT_POINT]
lsblk [OPTIONS] [DEVICE ...]
df [OPTIONS] [FILE ...]
```

分别从挂载表、块设备树和已挂载文件系统视角观察状态。三者共同完成“目录由谁提供、设备层级是什么、用户可见容量是多少”的验证。 **\*\*重要参数 / 形式\*\*** ``findmnt /srv/project``: 证明当前挂载源、目标和文件系统类型；不证明 ``fstab`` 持久性。 ``lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS``: 观察设备父子关系、块设备大小、文件系统和挂载点；不能替代 ``pvs`` 的 LVM 归属证据。 ``df -hT /srv/project``: 读取目标目录所在文件系统的类型、总量和使用率；不证明 VG 空闲 extent。

## [知识专题] 从设备到业务目录：容量链必须逐层成立

LVM 题目的对象不是抽象的“磁盘空间”，而是一条有明确身份和状态边界的容量链。正向创建时，容量从底层设备进入 LVM，再交给文件系统和挂载点；扩容时，应从业务目录反向找到对应 LV 和 VG。无论沿哪个方向工作，都不能让一层的成功代替下一层。

### ① [知识点] 设备存在不等于设备可写

`/dev/vdb1` 只是当前内核提供的块设备节点。它可能已包含文件系统、属于某个 VG、作为 swap 使用，或正在被挂载。写操作前至少要回答：

设备是否与题目指定对象一致？  
是否已有 FSTYPE、挂载点或 holder？  
是否已经出现在 pvs 中？  
题目是否明确允许覆盖其内容？

一个稳定的调查入口是：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdb  
findmnt --source /dev/vdb1  
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

设备信息存在冲突时应停止，不使用 `wipefs -a`、`pvccreate --force` 或删除现有 VG 作为默认“清理”步骤。分区创建和签名识别的完整机制留给第 22 章。

## ② [知识点] PV、VG、LV 分别承担身份、汇聚和分配

容量关系可以压缩为：

```
块设备 /dev/vdb1  
  ↓ pvccreate  
PV：可被 LVM 使用的容量身份  
  ↓ vgcreate / vgextend  
VG：由一个或多个 PV 汇成的容量池  
  ↓ lvcreate / lvextend  
LV：从 VG 获得容量的逻辑块设备
```

PV 不会自动进入任意 VG；新磁盘也不会自动成为 LV 的可用空间。若目标 VG 余量不足，新设备必须先成为 PV，再通过 `vgextend` 加入目标 VG，最后才能由 `lvextend` 消耗新增空闲 extent。

## ③ [知识点] 文件系统和挂载点属于 LVM 上层

LV 可以没有文件系统，也可以承载非文件系统用途。经典任务在新 LV 上创建 XFS 或 ext4，只是某一种上层用途。文件系统创建、标签、UUID 与增长机制的主归属是第 23 章；当前挂载与 `fstab` 持久性的主归属是第 24 章。

本章仍必须保留两个接口判断：

- `lvs` 变大只证明 LV 块设备增长；
- `findmnt` 与 `df` 才能证明业务目录仍指向目标文件系统，并看到新增容量。

## ④ [知识点] 从挂载点反向定位比从设备名正向猜更安全

题目若写“扩展 `/srv/reports`”，评分对象是业务目录，不是某个预设设备名。反向链如下：

```
findmnt /srv/reports
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
lvs -o lv_name,vg_name,lv_size,lv_path,devices
vgs -o vg_name,vg_free,vg_extent_size,vg_free_count
pvs -o pv_name,vg_name,pv_size,pv_free
```

每条命令缩小一次对象集合。若 `findmnt` 显示 NFS，应转入第 26 章；若源是普通分区，也不能假设其背后存在 VG。

## ⑤ [验证] 六层验收回答六个不同问题

设备层 `lsblk`：目标设备是否存在，大小、类型和当前上层关系是什么。

PV 层 `pvs`：设备是否获得 LVM 身份，属于哪个 VG。

VG 层 `vgs`：容量池总量、PE 和空闲 `extent` 是否符合预期。

LV 层 `lvs`：LV 路径、大小、类型和底层映射是否正确。

文件系统层 `df -hT`：文件系统是否识别新增空间。

挂载与数据层 `findmnt`、文件读取或校验和：业务路径和原数据是否仍正确。

[Cheatsheet] 创建从设备向上走；扩容从挂载点向下查。设备、PV、VG、LV、文件系统和挂载点是六个对象层，任何查询只证明其中一部分。

## [知识专题] PE、LE 与容量表达：先判断题目在说什么

LVM 同时支持容量单位、`extent` 数量和百分比表达。真正的难点不是记住四个参数，而是确定参照对象与时间语义：题目要求最终总量，还是在当前状态上增加；要求精确 `extent`，还是给出 MiB/GiB 容量；百分比相对哪个容量池。

### ① [知识点] PE 大小属于 VG，LE 是 LV 的分配视图

创建 VG 时确定 PE 大小。在线性 LV 的基础模型中，一个 LE 对应一个 PE，因此：

$$\text{LV 标称容量} = \text{LE 数量} \times \text{PE 大小}$$

例如 PE 为 16 MiB：

$$\begin{aligned} 50 \text{ LE} \times 16 \text{ MiB} &= 800 \text{ MiB} \\ 70 \text{ LE} \times 16 \text{ MiB} &= 1120 \text{ MiB} \end{aligned}$$

计算的价值是核对参数语义和目标终态，不是要求所有 `extent` 题都换算成容量后再使用 `-L`。

## ② [参数] 大写 `-L` 按容量，小写 `-l` 按 extent 或百分比

```
lvcreate -L 800M -n projectlv projectvg
lvcreate -l 50 -n projectlv projectvg
```

两条命令在 PE 为 16 MiB 时可能得到相同标称容量，但表达的评分对象不同。题目明确要求“50 个 extent”时使用 `-l 50`；题目只给“800 MiB”时使用 `-L 800M`。

## ③ [参数] 加号把目标总量改成相对增量

|                       |                      |
|-----------------------|----------------------|
| <code>-L 3G</code>    | 最终目标总量为 3 GiB        |
| <code>-L +512M</code> | 在当前基础上增加 512 MiB     |
| <code>-l 70</code>    | 最终目标总 extent 数为 70   |
| <code>-l +20</code>   | 在当前基础上增加 20 个 extent |

若当前 LV 已为 2 GiB，`lvextend -L 512M` 不是“增加 512 MiB”，而是给出更小的目标总量；`lvextend` 不执行缩小，命令会失败或明显不符合题意。此时正确形式是 `-L +512M`。

## ④ [参数] `%FREE` 的参照对象是 VG 当前空闲 extent

新建 LV：

```
lvcreate -l 100%FREE -n data1lv datavg
```

扩展已有 LV：

```
lvextend -l +100%FREE datavg/data1lv
```

第一条把当前全部空闲 extent 分配给新 LV；第二条把全部空闲 extent 追加给已有 LV。它们都会耗尽目标 VG 的余量，因此只有题目明确要求占满时才使用。生产环境通常需要为其他 LV、未来扩展或恢复策略保留空间。

## ⑤ [查询] 精确 extent 题不要用人类可读容量反推

```
vgs projectvg \
-o vg_name,vg_extent_size,vg_extent_count,vg_free_count

lvs --segments projectvg/projectlv \
-o lv_name,seg_start_pe,seg_size_pe,segtype,devices
```

`df -h` 的容量经过文件系统元数据和单位舍入，不能用来证明精确 LE 数量。普通单段线性 LV 可从 segment 视图核对 extent；复杂多段 LV 需要综合所有段，而不是只看第一行。

[Cheatsheet] `-L` 看容量, `-l` 看 extent; 无加号是目标总量, 有加号是相对增量; `%FREE` 以 VG 当前空闲 extent 为参照; 精确 extent 用 `vgs` 与 segment 报告验收。

## [操作专题] 用定制报告建立现状图, 而不是无目的地跑命令

调查的目标是决定下一步。先从业务路径和设备树确认对象, 再分别读取 PV、VG、LV 元数据。 `-o` 不是美化输出的选项, 而是把证据压缩到本次判断所需字段, 降低看错对象和忽略余量的风险。

### ① [查询] `findmnt` 与 `lsblk` 先回答“谁在使用谁”

```
findmnt /srv/project
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

`findmnt` 以挂载关系为中心, 适合从目录找到源; `lsblk` 以块设备树为中心, 适合观察磁盘、分区、LVM 映射和文件系统。它们能提示设备占用, 但不完整描述 LVM 元数据, 因此下一步仍要使用 `pvs`、`vgs`、`lvs`。

### ② [查询] `pvs` 证明 PV 身份与 VG 归属

```
pvs -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

``pv_name`` 哪个设备已经是 PV。

``vg_name`` 该 PV 属于哪个 VG; 空白通常表示尚未加入 VG。

``pv_size`` 该 PV 参与 LVM 管理的总容量。

``pv_free`` 从 PV 视角尚未分配给 LV 的容量。

``pv_attr`` PV 属性摘要, 用于发现状态差异。

`pv_free` 不是扩展任意 LV 的直接许可; 还必须确认该 PV 所属 VG 与目标 LV 的 VG 相同。

### ③ [查询] `vgs` 证明容量池是否有足够余量

```
vgs projectvg \
-o
vg_name,pv_count,lv_count,vg_size,vg_free,vg_extent_size,vg_extent_count,vg_free_count
```

``vg_free`` 适合判断 ``+512M``、``+2G`` 等容量增量能否满足。

``vg_extent_size`` 每个 PE 的大小, 是 extent 计算基础。

``vg_free_count`` 适合判断“增加 20 个 extent”能否满足。

`pv\_count` / `lv\_count` 帮助确认容量池成员与对象数量是否发生预期变化。

#### ④ [查询] `lvs` 证明 LV 的身份、路径、大小和映射

```
lvs projectvg/projectlv \  
-o lv_name,vg_name,lv_size,lv_path,lv_attr,segtype,devices
```

`lv_path` 给出上层工具应使用的真实设备路径；`devices` 展示底层映射；`lv_size` 只证明 LV 层大小。扩展后必须继续查看文件系统和挂载层。

#### ⑤ [操作方法] 把查询串成决策树

目标是业务目录

- `findmnt`: 源是什么、FSTYPE 是什么
- `lvs`: 它是否为 LV、属于哪个 VG
- `vgs`: 目标 VG 余量是否满足
- `pvs`: VG 的容量来自哪些设备
- 只有缺失证据得到回答后，才执行写操作

[Cheatsheet] `findmnt` 从目录找源；`lsblk` 看设备树；`pvs` 看设备身份与归属；`vgs` 看容量池和空闲 extent；`lvs` 看 LV 路径与大小。查询顺序应缩小对象，而不是堆积输出。

## [操作专题] 从确认空闲的设备创建 PV、VG 和 LV

创建链必须把每一次写入放在证据门之后。下面使用 `/dev/vdb1`、`projectvg`、`projectlv` 作为明确教学场景；真实环境必须以题目和当前查询结果替换。命令成功不是终点，每一层都要立即用对应报告验证。

#### ① [操作] 写入前建立设备安全门

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdb  
findmnt --source /dev/vdb1  
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

停止条件包括：设备名与题意不一致、已有需要保留的文件系统、当前正在挂载、已属于现有 VG、或题目没有授权覆盖。看到这些证据时，不继续执行 `pvccreate`，更不能默认清除签名。

#### ② [操作] `pvccreate` 只改变 PV 层

```
pvccreate /dev/vdb1  
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

第二条应证明 `/dev/vdb1` 已是 PV。此时 `vg_name` 仍可能为空，因为 PV 尚未加入 VG；这不是失败，而是对象链的中间状态。

### ③ [操作] `vgcreate -s` 建立容量池并确定 PE

```
vgcreate -s 16M projectvg /dev/vdb1
vgs projectvg \
  -o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_extent_count,vg_free_count
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free
```

`-s 16M` 把 PE 大小设为 16 MiB。验证必须同时看到：VG 名称正确、PV 已属于 `projectvg`、`vg_extent_size` 符合要求、VG 有可分配余量。

### ④ [操作] `lvcreate` 按题意分配容量

题目要求 50 个 extent：

```
lvcreate -l 50 -n projectlv projectvg
```

题目要求 800 MiB：

```
lvcreate -L 800M -n projectlv projectvg
```

创建后：

```
lvs projectvg/projectlv \
  -o lv_name,vg_name,lv_size,lv_path,lv_attr,segtype,devices
lvs --segments projectvg/projectlv \
  -o lv_name,seg_start_pe,seg_size_pe,segtype,devices
```

若评分目标是 extent，应重点核对 segment 的 extent 数和 VG 的 PE 大小，而不是只接受舍入后的 `lv_size`。

### ⑤ [边界] 文件系统与持久挂载只保留接口骨架

新 LV 仍是块设备。题目若要求 XFS 和持久挂载，需要调用第 23、24 章已经建立的能力：

```
mkfs.xfs /dev/projectvg/projectlv
mkdir -p /srv/project
blkid /dev/projectvg/projectlv
# 使用当前机器读取的真实 UUID 建立持久挂载记录
findmnt --verify
mount -a
findmnt /srv/project
```

本章不重新解释 `mkfs.xfs` 参数、UUID 生命周期或 `fstab` 六字段。这里强调的是：只有文件系统、当前挂载和持久配置均完成，业务目录才成立；不得把 LV 创建成功当作完整任务完成。

[Cheatsheet] 调查设备 → `pvccreate` → `pvs` → `vgcreate -s` → `vgs` → `lvcreate -L/-l -n` → `lvs`。每个写操作后立即证明对应对象层。

## [操作专题] VG 空间不足：先把新容量加入正确容量池

`lvextend` 只能消耗目标 VG 的空闲 PE。它不会搜索其他磁盘，不会自动初始化新设备，也不会借用其他 VG 的余量。空间不足时，稳定分支是“证明不足 → 调查候选设备 → 新建 PV → `vgextend` → 重新验证 VG → 再扩展 LV”。

### ① [诊断] 先证明不足发生在 VG 层

```
vgs projectvg \
-o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count
```

若任务要求增加 20 个 extent，而 `vg_free_count` 小于 20，当前 VG 的确无法满足。若 `vg_free_count` 足够，则空间不足错误可能来自选错 LV/VG、参数语义或其他限制，不应直接加入新设备。

### ② [操作] 对候选设备重新执行安全调查

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdc
findmnt --source /dev/vdc1
pvs /dev/vdc1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

新设备名不等于新设备状态。若 `/dev/vdc1` 已属于其他 VG，应停止并确认题意；不能为了当前任务强行夺取它。

### ③ [操作] 让设备先成为 PV，再加入目标 VG

```
pvccreate /dev/vdc1
vgextend projectvg /dev/vdc1
```

`vgextend` 参数顺序是目标 VG 在前、新 PV 在后。新磁盘不会直接作为 `lvextend` 参数，也不会因为 `pvccreate` 成功就自动属于 `projectvg`。

### ④ [验证] 重新证明成员关系和空闲 extent

```
pvs /dev/vdc1 -o pv_name,vg_name,pv_size,pv_free
vgs projectvg \
-o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_free_count
```

只有当 `/dev/vdc1` 已属于 `projectvg` 且 `vg_free_count` 或 `vg_free` 满足目标增量，才继续执行 `lvextend`。重复运行同一条空间不足命令不会自动修复容量池。

[Cheatsheet] `lvextend` 空间不足时先查 `vgs`；新设备先调查，再 `pvccreate` 和 `vgextend`；重新验证 VG 余量后才扩展 LV。

## [操作专题] 保留数据扩展 LV 与文件系统

安全扩容不是把命令缩成一行，而是把现有对象、容量来源、中间状态和原数据都纳入证据。扩容前先从挂载点确认文件系统类型、LV 和 VG；扩容后分别证明 LV、文件系统、挂载和数据。若使用 `-r`，仍不能省略分层验收。

### ① [操作] 建立扩容前基线

```
findmnt /srv/project
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path,devices
vgs projectvg -o vg_name,vg_free,vg_extent_size,vg_free_count
df -hT /srv/project
sha256sum /srv/project/marker.txt
```

基线至少回答：挂载点的源和 FSTYPE、当前 LV 大小、VG 余量、文件系统可见容量、需要保留的数据证据。若文件不存在或业务需要一致性快照，应使用符合场景的备份与一致性方法；单个校验和只证明被检查对象。

### ② [参数] 明确总量还是增量，再执行 `lvextend`

增加 20 个 extent：

```
lvextend -l +20 projectvg/projectlv
```

增加 512 MiB：

```
lvextend -L +512M datavg/reportslv
```

扩展到总大小 3 GiB：

```
lvextend -L 3G datavg/reportslv
```

执行前再次确认 VG 余量；执行后立即运行 `lvs`，记录 LV 的实际中间状态。不要在不清楚结果时重复追加相同增量，否则可能得到双倍扩容。

### ③ [操作] `-r` 联动两个状态层，但不是原子承诺

```
lvextend -l +20 -r projectvg/projectlv
```

`-r / --resizefs` 在扩大 LV 后调用文件系统调整工具。它能减少手工步骤，但不意味着任意文件系统都支持、也不保证失败时自动恢复原大小。命令报错后必须重新查询：

```
lvs projectvg/projectlv -o lv_name,lv_size,lv_path
findmnt /srv/project
df -hT /srv/project
```

可能出现的部分终态是“LV 已增长、文件系统未增长”。此时最小修复是确认 FSTYPE 和正确增长入口，而不是再次执行相同的 `lvextend +增量`。

#### ④ [操作] 手工增长已挂载 XFS

底层 LV 已扩大且 `findmnt` 证明 `/srv/project` 是目标 XFS：

```
xfs_growfs /srv/project
```

增长入口使用已挂载文件系统的挂载点。操作后：

```
df -hT /srv/project
findmnt /srv/project
```

XFS 在 RHEL 9 主线中不可原地缩小。若目标是减少容量，应重新设计迁移方案，不能先 `lvreduce` 再尝试修复文件系统。

#### ⑤ [操作] 手工增长 ext4

底层 LV 已扩大且确认文件系统为 ext4：

```
resize2fs /dev/datavg/reportslv
```

不指定大小时，`resize2fs` 通常增长到当前块设备可用上限。操作后比较：

```
lvs datavg/reportslv -o lv_name,lv_size,lv_path
df -hT /srv/reports
findmnt /srv/reports
```

ext4 支持缩小不等于本章提供通用缩容答案；缩小需要离线条件、文件系统先行和更严格的容量计算，主归属在第 23 章。

#### ⑥ [验证] 扩容后的证据必须跨越六层

VG 余量`vgs`：新增容量是否来自正确 VG，剩余 extent 是否合理。

LV 大小`lvs`：LV 是否按目标总量或增量增长。

文件系统容量`df -hT`：上层是否真正使用新增块空间。

当前挂载`findmnt`：业务目录是否仍由原目标 LV 提供。

设备映射`lsblk`：设备树、FSTYPE 和挂载关系是否一致。

已有数据读取文件或比较校验和：扩容是否保留指定数据证据。

[Cheatsheet] 基线 → 确认 VG 余量 → `lvextend` 表达正确增量 → `-r` 或匹配的文件系统增长工具 → `lvs`、`df`、`findmnt`、数据证据分层验收。

## [诊断专题] 容量没有按预期出现：让下一条证据定位断裂层

诊断不从“再试一个命令”开始，而从症状所在层开始。每个路径都按“症状 → 当前证据 → 假设 → 下一条有区分度的证据 → 最小修复 → 再验证”推进。

### ① [诊断] `lvs` 已增大，`df` 完全没变化

当前证据：LV 层增长，文件系统层未体现。优先假设：文件系统增长未执行、执行失败，或目标挂载点/设备选错。下一条证据：

```
findmnt /srv/project
lsblk -f
lvs projectvg/projectlv -o lv_name,lv_size,lv_path
df -hT /srv/project
```

确认 FSTYPE 后，对正确对象执行 `xfs_growfs <MOUNT_POINT>` 或 `resize2fs <LV_PATH>`。不要再次追加相同 LV 增量。

### ② [诊断] `lvextend` 报可用空间不足

当前证据：扩展请求无法从目标 VG 分配。下一条证据：

```
vgs <VG> -o vg_name,vg_free,vg_extent_size,vg_free_count
```

若余量不足，调查候选设备并执行 `pvccreate → vgextend → 重新验证`；若余量足够，检查是否选错 VG/LV、增量是否超过预期或对象状态是否异常。

### ③ [诊断] 目录存在且可写，但 `findmnt` 没有结果

当前证据：目录只是当前目录树中的普通目录，目标文件系统没有挂载。风险：继续写入会占用承载该目录的上层文件系统，常见是根文件系统。下一条证据：

```
findmnt -T /srv/project
df -hT /srv/project
```

进入第 24 章检查当前挂载和持久配置。不要用“目录里已经有文件”证明挂载成功。

#### ④ [诊断] `pvccreate` 发现已有文件系统签名或 PV

当前证据：设备不是无状态空白对象。下一条证据：

```
lsblk -f /dev/vdb1
findmnt --source /dev/vdb1
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

重新核对题目与设备身份。除非题目明确要求清除且数据无需保留，否则不使用强制选项继续。

#### ⑤ [诊断] `vgextend` 提示设备属于其他 VG

设备已有容量归属。下一条证据是：

```
pvs /dev/vdc1 -o pv_name,vg_name,pv_size,pv_free
vgs -o vg_name,pv_count,lv_count,vg_size,vg_free
```

确认该 VG 是否承载其他 LV 和数据。默认停止，不执行 `vgreduce`、`pvremove` 或强制迁移来“抢”容量。

#### ⑥ [诊断] `lvs` 与 `df` 只差少量容量

少量差异通常来自文件系统元数据、保留空间、单位换算和舍入。判断是否异常，应比较扩容前后趋势，并确认目标是否达到允许范围；精确 extent 验收使用 `vgs` 和 `segment` 字段，不使用 `df -h` 反推。

#### ⑦ [诊断] `lvextend -r` 返回错误，终态不清楚

不要假设整个操作回滚。先查询：

```
lvs <VG>/<LV> -o lv_name,lv_size,lv_path
findmnt <MOUNT_POINT>
df -hT <MOUNT_POINT>
```

若 LV 已增长而文件系统未增长，按 `FSTYPE` 执行最小文件系统修复；若 LV 没增长，则回到 VG 余量和命令目标。任何情况下都不应盲目再次执行同一增量。

[Cheatsheet] 先判定断裂层：VG、LV、文件系统、挂载或数据。下一条证据必须能区分假设；最小修复只改变出错层，随后重新完成跨层验收。

## [诊断专题] 删除、缩容和强制覆盖：不能把高风险动作压成快捷表

扩容通常只需要增加可用容量，而缩容和删除会改变数据可达边界。它们不是对称操作。面对空间规划错误、选错设备或已有签名时，最重要的能力是知道何时停止，而不是能背出最多的破坏性命令。

危险边界：先缩小 LV 会直接截断块设备尾部。

若文件系统仍在使用被截断区域，数据和元数据可能立即损坏。任何缩容都必须由文件系统能力和实际占用先决定，不能从 `lvreduce` 开始。

### ① [边界] XFS 不可原地缩小

RHEL 9 主线中的 XFS 只支持增长。题目若要求减少 XFS 所在存储，应将需求识别为迁移设计：建立新的较小文件系统、复制并验证数据、切换挂载，再在确认退役后处理旧对象。本章不把这条高风险迁移简化为考试命令串。

### ② [边界] ext4 可缩小，不等于可以机械照抄

ext4 的缩小通常需要卸载、文件系统检查、先缩文件系统再缩底层 LV，并准确保留安全余量。完整流程属于第 23 章的文件系统容量管理；本章只保留原则：文件系统必须先安全缩小，LV 才能随后缩小。

### ③ [边界] 删除命令只服务于明确退役

`lvremove`、`vgremove`、`pvremove` 会逐层解除对象和元数据。只有在以下条件都明确时才讨论：目标对象身份无歧义、没有挂载或活动依赖、数据已经按要求保留、题目明确要求退役。删除不是修复创建错误的默认方法。

### ④ [边界] 已有签名不是要求使用 `--force`

工具检测到已有文件系统、PV 或 VG 归属，是阻止数据破坏的证据。稳定处理是查明来源和题意，必要时选择正确设备；只有明确的清除任务才进入签名移除流程，而该流程主归属第 22 章。

### ⑤ [边界] 校验和是局部证据，不是完整备份

经典任务通过 `marker.txt` 或 `sha256sum` 证明指定文件在扩容前后保持一致。它能排除“重新格式化后重新写同名空文件”等错误路径，但不能证明整个数据库或应用的一致性。生产变更仍需要业务级备份、停写、快照或一致性控制。

[Cheatsheet] XFS 不缩；ext4 缩容先文件系统、后 LV，且不在本章给出机械答案；删除和 `force` 只在对象、数据和退役要求全部明确时使用。

## [知识专题] 从考试容量题迁移到生产变更

---

考试环境用具体设备名和容量训练对象模型；生产环境还要面对业务一致性、容量增长速度、共享 VG、回退与变更窗口。迁移的重点不是增加更多命令，而是把调查、停止条件和验收写成可复核流程。

### ① [工作迁移] 保存扩容前基线和变更对象

建议至少记录：挂载源、FSTYPE、LV 路径和大小、VG 总量与余量、PV 成员、业务数据验证方法。基线让“操作后变了什么”可比较，也能防止多次执行相同增量。

## ② [工作迁移] 不默认吃满 VG

**100%FREE** 在考试中可能是明确要求，在生产中却会消耗快照、其他 LV 和未来增长的空间。容量设计应说明保留量、增长阈值和下一次扩容来源，而不是只追求当前最大化。

## ③ [工作迁移] 把“LV 增长”和“业务可用”写成两个验收项

变更单中至少分别列出：

LVM 终态：目标 LV 大小、VG 剩余容量、PV 成员正确

业务终态：文件系统容量增长、挂载源不变、应用数据可读、必要功能正常

这能避免底层成功被误写成业务完成。

## ④ [工作迁移] 明确回退边界

扩容本身通常不可通过简单缩容无风险撤销；新 PV 加入 VG 后也可能开始承载数据。生产变更的回退应更多依赖备份、快照、应用切换和预先设计，而不是在失败后临时执行 `lvreduce` 或删除对象。

[Cheatsheet] 生产扩容：保存基线、说明容量来源、保留 VG 余量、把 LVM 与业务终态分开验收、提前声明不可逆边界。

# [经典任务] 创建指定 extent 的项目卷并保留数据扩展

## 环境与当前状态

- `/dev/vdb1` 和 `/dev/vdc1` 是题目提供的候选分区；题目说明其内容无需保留，但仍须先查询确认身份与占用。
- `projectvg`、`projectlv` 和 `/srv/project` 尚不存在。
- 文件系统与持久挂载能力已经在第 23、24 章学习。

## 目标终态

1. 使用 `/dev/vdb1` 创建 PV。
2. 创建 `projectvg`，PE 大小为 16 MiB。
3. 创建 `projectlv`，初始大小为 50 个 extent。
4. 在 LV 上创建 XFS，并通过真实 UUID 持久挂载到 `/srv/project`。
5. 写入 `/srv/project/marker.txt`，内容为 `keep-this-data`。
6. 在保留该文件的前提下，再为 LV 增加 20 个 extent，并让 XFS 使用新增空间。
7. 若 `projectvg` 的空闲 extent 不足，先把 `/dev/vdc1` 安全加入容量池。

## 限制条件

- 不重新格式化已有 XFS。
- 不删除并重建 LV。
- 不缩小其他 LV。
- 不使用 `--force`、`-ff`、无调查的签名清除或删除命令。
- 不把目录存在、命令返回成功或 `lvs` 单层增长当作完整验收。

## 验收矩阵

| 层次    | 必须证明的终态                             | 推荐证据                                           |
|-------|-------------------------------------|------------------------------------------------|
| 设备/PV | 候选设备身份正确，PV 归属正确                    | <code>lsblk</code> 、 <code>pvs</code>          |
| VG    | PE 为 16 MiB，容量来源正确                  | <code>vgs</code>                               |
| LV    | 初始 50 extent，最终 70 extent           | <code>lvs --segments</code> 、 <code>lvs</code> |
| 文件系统  | XFS 使用新增块空间                         | <code>df -hT</code>                            |
| 挂载    | <code>/srv/project</code> 当前源为目标 LV | <code>findmnt</code>                           |
| 持久性   | 持久挂载条目使用真实 UUID 且可验证                | 第 24 章验证链                                      |
| 数据    | <code>marker.txt</code> 内容或校验和保持一致  | <code>cat</code> 、 <code>sha256sum</code>      |

请先独立完成。参考解答从新页开始。

## [参考解答] 从设备安全门到分层验收

### ① 调查两个候选设备

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdb /dev/vdc
findmnt --source /dev/vdb1
findmnt --source /dev/vdc1
pvs -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

只在证据与题意一致、设备没有需保留数据或冲突归属时继续。题目说明“内容无需保留”不能替代设备名核对。

### ② 创建 PV 和指定 PE 的 VG

```
pvcreeate /dev/vdb1
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr

vgcreate -s 16M projectvg /dev/vdb1
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free
vgs projectvg \
    -o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_extent_count,vg_free_count
```

参数选择：-s 16M 直接表达 PE 目标；验证应看到 /dev/vdb1 已属于 projectvg，vg\_extent\_size 为 16 MiB。

### ③ 按 extent 创建 LV

```
lvcreate -l 50 -n projectlv projectvg
lvs projectvg/projectlv \
    -o lv_name,vg_name,lv_size,lv_path,lv_attr,segtype,devices
lvs --segments projectvg/projectlv \
    -o lv_name,seg_start_pe,seg_size_pe,segtype,devices
```

题目按 extent 评分，因此使用 -l 50，不先换算成 -L 800M。在线性单段场景中，segment 报告应体现 50 个 extent。

### ④ 调用文件系统与持久挂载能力

```
mkfs.xfs /dev/projectvg/projectlv
mkdir -p /srv/project
blkid /dev/projectvg/projectlv
```

使用 blkid 读取的真实 UUID 按第 24 章方法建立持久挂载记录，然后在重启前验证：

```
findmnt --verify
mount -a
findmnt /srv/project
df -hT /srv/project
```

这里不复述 `fstab` 六字段；关键边界是不能抄写示例 UUID，也不能仅凭目录存在宣布完成。

## ⑤ 建立数据保留证据

```
printf '%s\n' 'keep-this-data' > /srv/project/marker.txt
cat /srv/project/marker.txt
sha256sum /srv/project/marker.txt
```

记录扩容前校验值。该证据只覆盖指定文件，不替代完整备份。

## ⑥ 判断 VG 是否有 20 个空闲 extent

```
vgs projectvg -o vg_name,vg_free,vg_extent_size,vg_free_count
```

若 `vg_free_count` 至少为 20，直接进入扩展。若不足，先重新调查 `/dev/vdc1`，再执行：

```
pvccreate /dev/vdc1
vgextend projectvg /dev/vdc1
pvs /dev/vdc1 -o pv_name,vg_name,pv_size,pv_free
vgs projectvg \
    -o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_free_count
```

只有新 PV 已属于 `projectvg` 且余量满足，才继续。

## ⑦ 增加 20 个 extent 并扩大 XFS

推荐联动形式：

```
lvextend -l +20 -r projectvg/projectlv
```

参数选择：小写 `-l` 按 extent；`+20` 表示增加量；`-r` 继续调整受支持文件系统。若命令报错，不重复执行，先查询 LV 和文件系统实际状态。

手工分离形式也成立：

```
lvextend -l +20 projectvg/projectlv
xfs_growfs /srv/project
```

## ⑧ 最终分层验证

```
pvs -o pv_name,vg_name,pv_size,pv_free
vgs projectvg \
  -o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_extent_count,vg_free_count
lvs projectvg/projectlv \
  -o lv_name,vg_name,lv_size,lv_path,lv_attr,segtype,devices
lvs --segments projectvg/projectlv \
  -o lv_name,seg_start_pe,seg_size_pe,segtype,devices
findmnt /srv/project
df -hT /srv/project
cat /srv/project/marker.txt
sha256sum /srv/project/marker.txt
```

最终判断：LV 应由 50 extent 增加到 70 extent；PE 为 16 MiB 时标称容量为 1120 MiB。`df` 可能因文件系统元数据和舍入略小，但应相对扩容前明显增长；挂载源与校验值应保持一致。

### 典型错误分支

- `lvextend` 报空间不足：先查 `vgs`，不要反复执行。
- `lvs` 已增长而 `df` 未变：确认 FSTYPE，执行 `xfs_growfs`，不要再次追加 20 extent。
- `findmnt` 无结果：进入第 24 章修复挂载，不把目录中的文件当作目标文件系统数据。
- `pvccreate` 检测到已有签名：停止并重新确认设备，禁止默认 force。

# [经典任务] 保留 ext4 数据扩展现有报告卷

## 环境与当前状态

- `/srv/reports` 当前已挂载，文件系统类型为 `ext4`。
- 挂载源应通过查询确认，预期为 `/dev/datavg/reportslv`。
- LV 当前约为 2 GiB。
- `/srv/reports/report.db` 已存在，内容必须保持不变。
- `datavg` 是否有足够余量未知；`/dev/vdd1` 是题目提供的候选扩容设备。

## 目标终态

1. 把 LV 增加 512 MiB，而不是设置为总大小 512 MiB。
2. 本任务不使用 `-r`，要求分别扩展 LV 和 `ext4`。
3. 若 VG 余量不足，安全地把 `/dev/vdd1` 加入 `datavg`。
4. 最终挂载源仍为原 LV，文件系统容量增加，`report.db` 校验和不变。

## 限制条件

- 不重新格式化 `ext4`。
- 不删除或重建 LV。
- 不使用 `force` 或无调查的签名清除。
- 不以 `lvs` 增长代替文件系统和数据验收。

## 验收矩阵

| 层次   | 目标                                                  | 证据                                           |
|------|-----------------------------------------------------|----------------------------------------------|
| 映射   | <code>/srv/reports</code> 指向预期 <code>ext4</code> LV | <code>findmnt</code> 、 <code>lsblk -f</code> |
| VG   | 有至少 512 MiB 可分配余量                                   | <code>vgs</code>                             |
| LV   | 当前基础上增加 512 MiB                                     | <code>lvs</code>                             |
| 文件系统 | <code>ext4</code> 使用新增空间                            | <code>df -hT</code>                          |
| 数据   | <code>report.db</code> 校验和不变                        | <code>sha256sum</code>                       |

请先独立完成。参考解答从新页开始。

## [参考解答] 分开观察 LV 与 ext4 的增长

### ① 建立映射、容量和数据基线

```
findmnt /srv/reports
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
lvs datavg/reportslv -o lv_name,vg_name,lv_size,lv_path,devices
vgs datavg -o vg_name,vg_free,vg_extent_size,vg_free_count
df -hT /srv/reports
sha256sum /srv/reports/report.db
```

先确认实际挂载源和 ext4 类型。若源不是目标 LV，停止并调查；不能按题面预期盲目操作设备。

### ② 确认或补充 VG 余量

若 `vg_free` 足够 512 MiB，直接继续。若不足，调查候选设备：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdd
findmnt --source /dev/vdd1
pvs /dev/vdd1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

证据允许后：

```
pvcreeate /dev/vdd1
vgextend datavg /dev/vdd1
pvs /dev/vdd1 -o pv_name,vg_name,pv_size,pv_free
vgs datavg -o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_free_count
```

### ③ 只扩展 LV，观察预期中间状态

```
lvextend -L +512M datavg/reportslv
lvs datavg/reportslv -o lv_name,vg_name,lv_size,lv_path
findmnt /srv/reports
df -hT /srv/reports
```

参数选择：`+512M` 表示在当前基础上增加。此时 `lvs` 应增长，而 `df` 可能尚未变化；这是本任务刻意保留的中间状态，用于证明 LV 和文件系统是两个对象层。

### ④ 扩展 ext4

```
resize2fs /dev/datavg/reportslv
```

设备路径应来自 `lvs -o lv_path` 或当前映射证据，而不是未经确认的手工拼接。

## ⑤ 最终验证

```
lvs datavg/reportslv -o lv_name,vg_name,lv_size,lv_path
df -hT /srv/reports
findmnt /srv/reports
sha256sum /srv/reports/report.db
```

LV 应比基线增加 512 MiB，ext4 可见容量应增长，挂载源不变，校验和一致。

### 典型错误分支

- 写成 `lvextend -L 512M`：这是目标总量语义，不是增量；应停止并改为 `+512M`。
- `lvs` 增长、`df` 不变：执行正确的 `resize2fs`，不重复扩展 LV。
- `resize2fs` 报设备或类型错误：重新核对 `findmnt`、`lsblk -f` 和 `lv_path`。
- 校验和变化：不能把容量增长视为完成，应保留现场并调查数据或挂载源变化。

## [本章收束] 容量链也是证据链

LVM 的核心不是背三个缩写，而是知道容量在每一层如何获得身份、被汇聚、被分配并最终交付给业务目录。创建题从确认设备开始，逐层建立 PV、VG、LV；扩容题从挂载点反向确认文件系统、LV 和 VG，先证明容量来源，再改变对象；诊断题则比较各层证据，找出哪一层没有成立或没有同步。

最稳定的工作路径是：

目标目录由谁提供

- 对应哪个文件系统和 LV
- LV 属于哪个 VG
- VG 是否有足够空闲 extent
- 参数表达的是总量还是增量
- LV 与文件系统是否都增长
- 挂载和原数据是否仍正确

## 主要判断表

| 现象或目标                                   | 先看什么                                                     | 下一步原则                                                |
|-----------------------------------------|----------------------------------------------------------|------------------------------------------------------|
| 创建指定 extent 的 LV                        | <code>vg_extent_size</code> 、 <code>vg_free_count</code> | 用 <code>-l</code> 直接表达 extent                        |
| 增加 512 MiB                              | 当前 LV、 <code>vg_free</code>                              | 使用 <code>-L +512M</code>                             |
| <code>lvextend</code> 空间不足              | <code>vgs</code> 余量                                      | 新设备先 <code>pvccreate</code> 、再 <code>vgextend</code> |
| <code>lvs</code> 增长、 <code>df</code> 不变 | FSTYPE、挂载源                                               | 只修复文件系统层，不重复增量                                       |
| 目录存在但未挂载                                | <code>findmnt -T</code>                                  | 转入第 24 章修复挂载                                         |

| 现象或目标         | 先看什么                                     | 下一步原则              |
|---------------|------------------------------------------|--------------------|
| 设备已有签名或 VG 归属 | <code>lsblk -f</code> 、 <code>pvs</code> | 停止并重新确认，默认不 force  |
| 要求缩小 XFS      | 文件系统类型                                   | 识别为不支持的原地缩小需求，设计迁移 |

## 本章工作方法

---

写操作前重新确认设备身份和数据保留要求，查询只保留会改变下一步决策的字段；对 `-L`、`-l`、加号和 `%FREE` 明确参照对象，每改变一层就立即使用该层报告验证。最终验收必须跨越 LV、文件系统、挂载和数据层，删除、缩容和强制覆盖不作为空间问题的默认修复。

## 向下一章交接

---

第 26 章《NFS 客户端与远程文件系统》会处理远端源、NFSv3/v4 必要差异、客户端挂载与身份映射。若 `findmnt` 显示业务目录来自 NFS，本章的 PV/VG/LV 容量链不再适用；应保留当前证据，转入远程文件系统的服务端容量、导出与客户端访问链。